

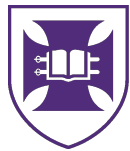
Algorithms and Data Structures

COMP3506/7505

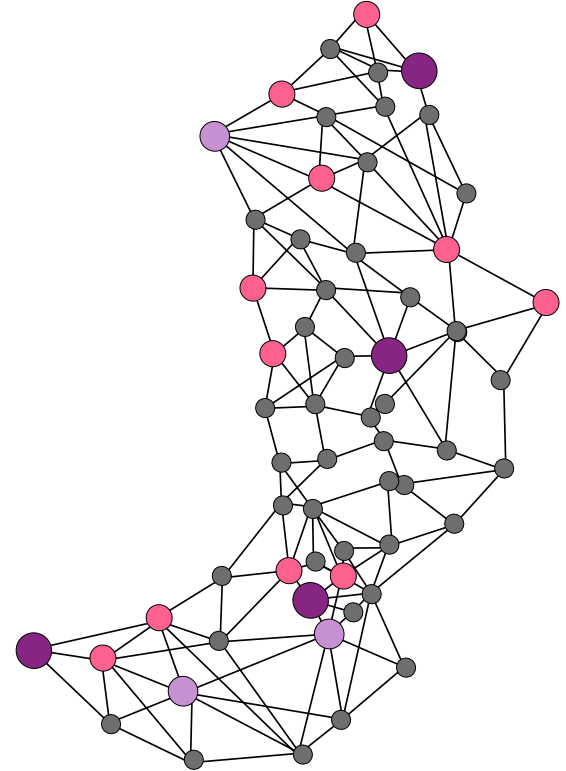
Semester 2, 2023

Week 13: Course Review

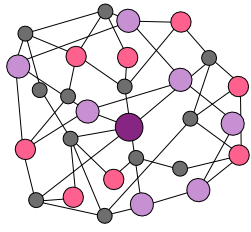
Dr Joel Mackenzie



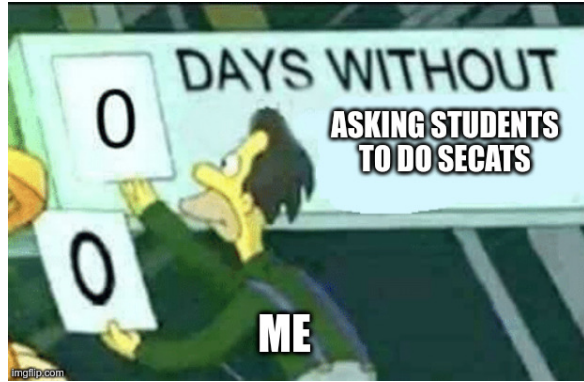
THE UNIVERSITY
OF QUEENSLAND
AUSTRALIA



Agenda

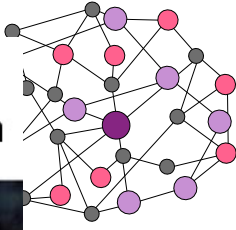


- ♦ **Exam format and Information**
- ♦ Whirlwind tour of Algorithms and Data Structures
- ♦ Sample Problems / Selected Revision



Joel forcing
us to do secats

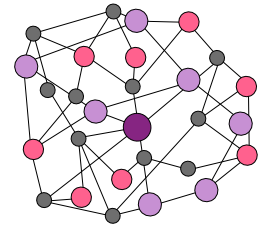
innocent students
trying to study for the exam



SECaTs are now open

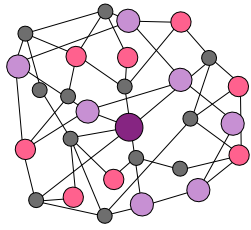
- ♦ Close at **11:59pm on Friday 3 November 2022**
- ♦ **Link provided via email or <https://eval.uq.edu.au/>**
- ♦ Feedback is highly valued!

Exam Details



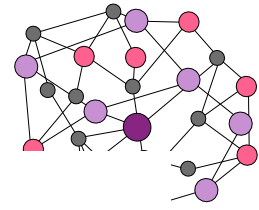
- ♦ **Friday November 10th at 8:00 AM** (sorry)
 - ♦ Please check your personal exam timetable
 - ♦ Please check SI-net for official information
- ♦ 2 hour exam, with 10 minutes reading time
- ♦ 100 marks, 120 minutes → ~1 mark per minute
- ♦ Calculators allowed but not likely to be useful
 - ♦ UQ approved calculators only, no memory
- ♦ One two-sided page of notes is allowed
- ♦ Hurdle: 50% required

Exam Format



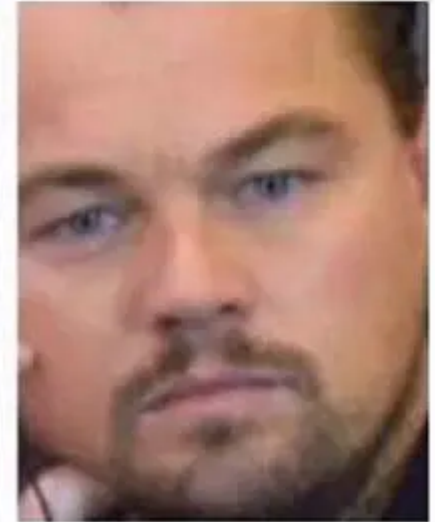
- ♦ Different from previous years
 - ♦ **First part:** 20 marks, True/False
 - ♦ 1 mark for the answer, and 1 mark for an explanation
 - ♦ 10 questions in total
 - ♦ **Second part:** 8 short answer questions
 - ♦ One python programming question
 - ♦ Pseudocode questions
 - ♦ Worked problems
 - ♦ **Last question:** Extended problem/open problem
 - ♦ See sample provided

Exam Survival Guide

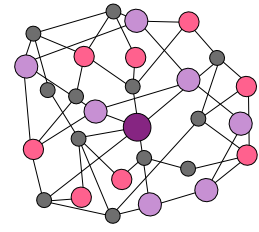


- Line of reasoning will get you a long way
- Practice previous exams
 - ◆ Practice under time pressure
 - ◆ Use a greedy approach
 - ◆ Find weak spots
- Look at the examples and questions from tutorials
- Look at the examples and questions from the lectures
- Revise quizzes
- **MAKE YOUR OWN QUESTIONS**

When you had months to prepare for your exams but you still left it all until the night before

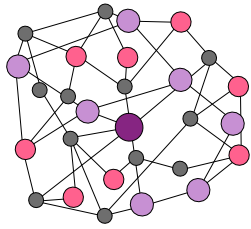


Agenda



- ♦ Exam format and Information
- ♦ **Whirlwind tour of Algorithms and Data Structures**
- ♦ Sample Problems

COMP3506/7505 Modules



Computational Model and Asymptotic Analysis

Recursion, Comparison-Based sorting

Linear sorting, lists (amortization)/arrays/linked structures

Stacks, queues, tree basics

Priority queues, heaps, adaptable priority queues, [maps, sets, multisets, multimaps] ADTs

Hashing, hash tables, binary search trees

Balanced trees (AVL, Splay, 2-4, red-black, multiway)

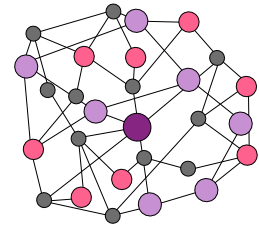
Graph basics, traversal, directed graphs

Shortest paths, minimum spanning trees, attacking graph problems

Strings, tries, pattern matching, suffix arrays

Huffman, ~~Lempel-Ziv~~

COMP3506/7505 Modules



Computational Model and Asymptotic Analysis

Recursion, Comparison-Based sorting

Linear time (amortization)/arrays/linked structures

Stacks, queues, [maps, sets, multisets, priority queues, ...]

Pr...

multimaps, ...

Hashing, hash tables, ...

Balanced trees (AVL, Splay, ...)

Graph basics, traversal, directed graphs

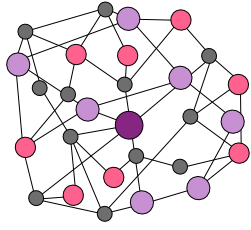
Shortest paths, minimum spanning trees, attacking

Strings, tries, pattern matching, suffix arrays

Huffman, Lempel-Ziv

Everything from week 1 to week 11 is assessable including quiz questions, assignment, lecture, tutorial, ...

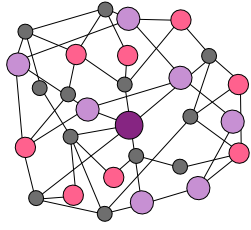
Week 1 and 2: Analysis



Need to Know

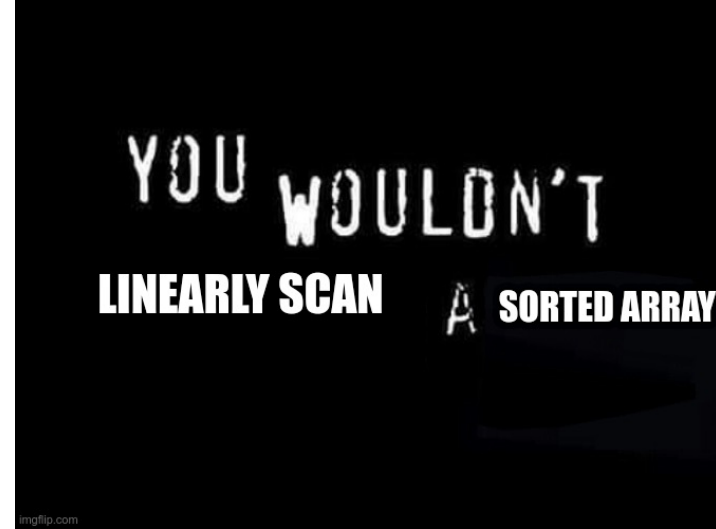
- Formal definition of big-O, Omega, Theta and how they relate to one another
- Worst, average, best-case analysis and their differences
- Classes of growth and how they relate to one another
- How to model and count operations
- How to model and calculate time/space complexity

Week 1 and 2: Recursion

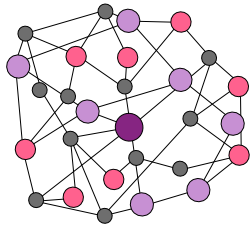


Need to Know

- Types of recursion
- Modelling recurrences
- **Solving recurrences**
- Divide-and-Conquer (binary search, mergesort, ...)



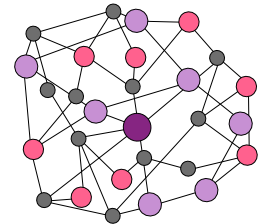
Week 2 and 3: Sorting



Need to Know

- Basic comparison-based: bubble sort, insertion sort, selection sort
- Better comparison-based: mergesort, quicksort
- Comparison free (linear sorting): bucket and radix sort
- **Runtime and space complexity of these algorithms**
- When to choose or apply these different algorithms

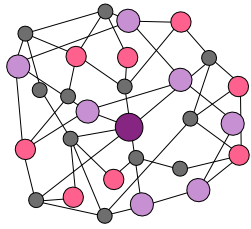
Week 3 and 4: Linear Structures



Need to Know

- Link-based structures (singly, doubly linked lists + augmentation)
- Arrays, ArrayLists (Lists, ExtensibleLists, ...) including **amortization**
- Positional lists
 - ◆ Come in handy when augmenting other data structures like adaptive PQs for example
- Stacks and queues

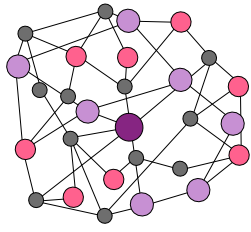
Week 4: Tree Basics



Need to Know

- Terminology and properties
- Applications of trees
- Binary trees
- Traversals (pre/in/post-order, Euler tours)
- Linked vs array-based representations

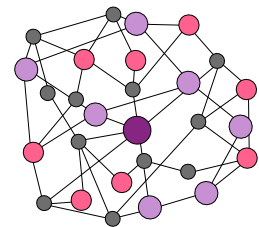
Week 5: Priority Queues



Need to Know

- ♦ PQ implementations
 - ◆ Unsorted list, sorted list, heap (tree and array-based)
 - ◆ Focus on array-based min-heap
- ♦ Heapsort and unified sorting model
- ♦ Applications/properties/operations of heaps/queues

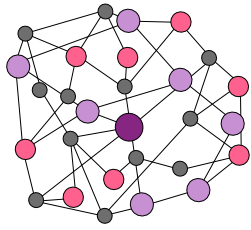
Week 5: Adaptable PQs, Maps, Sets, ...



Need to Know

- ♦ Adaptable PQ (can have keys updated)
- ♦ ADTs: Map, Set, Multimap, Multiset
- ♦ Basic ideas about their implementation
 - ♦ Think for 2 seconds...

Week 6: Hashing

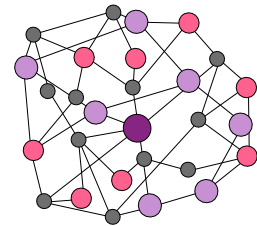


Need to know

- Hashtable applications, operations, properties
- Hash codes and compression functions
- Collision Handling:
 - ◆ Linear/Quadratic probing
 - ◆ Double hashing
 - ◆ Separate Chaining
- Hash tables: Sets/Unordered Maps



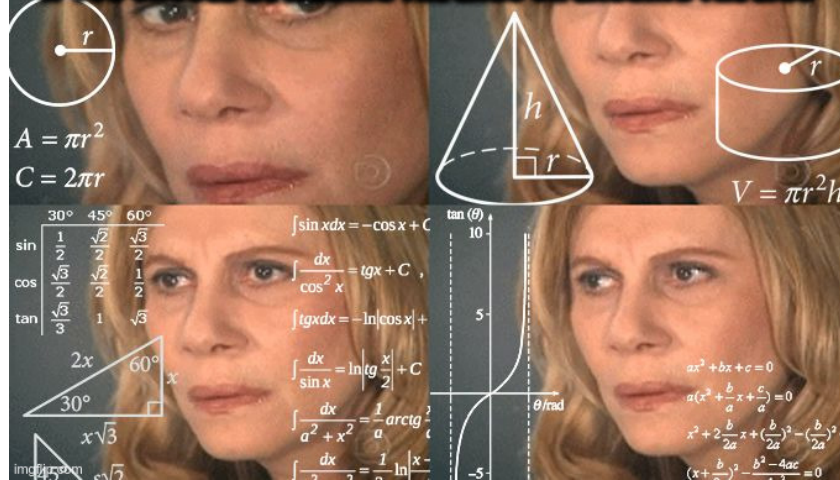
Week 6 and 7: Ordered Maps



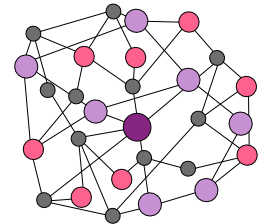
Need to know

- ♦ Properties, applications, and operations:
 - ♦ Binary Search Tree
 - ♦ AVL Tree
 - ♦ Splay Tree
- ♦ “Gist” of:
 - ♦ Multi-way trees

WHEN YOUR TREE IS UNBALANCED BUT YOU CANT REMEMBER
IF YOU SHOULD BE ZIGGING THE ZAGS OR ZAGGING THE ZIGS



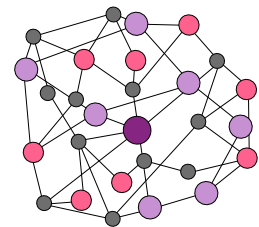
Week 8 and 9: Graphs



Need to know

- ♦ Properties, applications, terminology
- ♦ How to model real-world problems as graph problems
- ♦ Graph implementations:
 - ♦ Edge list
 - ♦ **Adjacency list**
 - ♦ Adjacency matrix

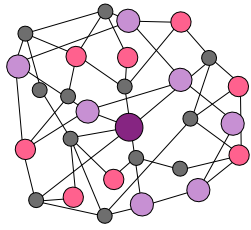
Week 8 and 9: Graphs



Need to know

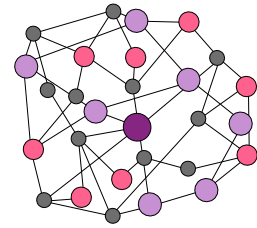
- ♦ DFS and BFS (properties, applications, implementations)
- ♦ Reachability, connectivity
- ♦ Topological sorting
- ♦ Shortest paths (undirected, dag, weights, ...)
- ♦ Minimum spanning trees

Week 10 and 11: Strings, Patterns



Need to know

- ♦ Pattern matching algorithms
 - ♦ Brute force, Boyer-Moore, KMP
- ♦ Tries and compressed tries
- ♦ Suffix tries → Suffix arrays
- ♦ Huffman coding
 - ♦ Building a Huffman code, applying a Huffman code



What are your weak spots?

"I'm going to study at 5."

"I'm going to study at 6."

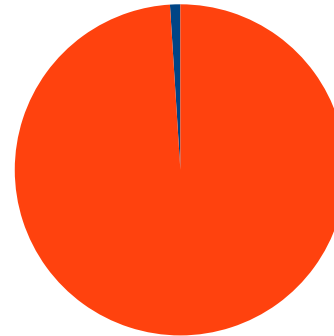
"I'm going to study at 7."

"I'm going to study at 8."

"I'm going to study at 9."



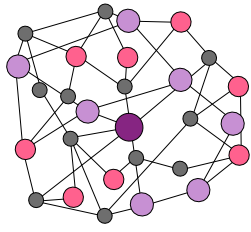
Things I do in the week before the exam



■ Study

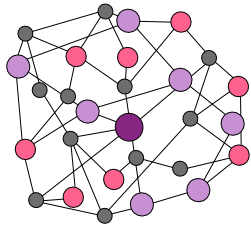
■ Tell People I have to Study

Agenda



- ♦ Exam format and Information
- ♦ Whirlwind tour of Algorithms and Data Structures
- ♦ **Sample Problems / Selected Revision**

Reminder: Analysis



Express algorithm in pseudocode

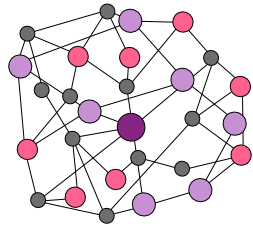
Model the cost by counting primitive operations

- As governed by Word-RAM

Describe the cost of the algorithm as a function of n , the size of the input to the algorithm

Perform asymptotic analysis and express the runtime in asymptotic notation

Reminder: Big-O, Omega, Theta

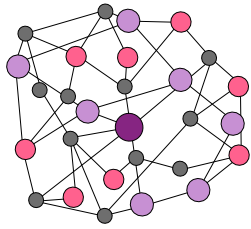


$f(n)$ is $O(g(n))$ if there exist positive constants c and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$

$f(n)$ is $\Omega(g(n))$ if there exist positive constants c and n_0 such that $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$

$f(n)$ is $\Theta(g(n))$ if there exist positive constants c_1 , c_2 , and n_0 such that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$

Simplification Rules

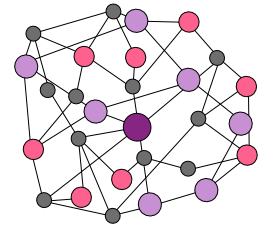


- Drop constants
- Drop lower-order terms

Remember: Big-O is just an *upper bound* (and big-Omega is just a *lower bound*)

- Most algorithms are in $O(n!)$
- All algorithms that do at least one operation are $\Omega(1)$

Rates of Growth



$O(1)$

Constant

$O(\log n)$

Logarithmic

$O(n)$

Linear

$O(n \log n)$

Super linear

$O(n^2)$

Quadratic

$O(n^3)$

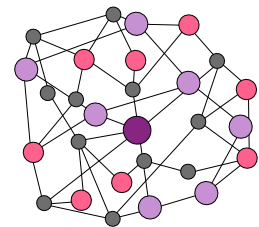
Cubic

$O(2^n)$

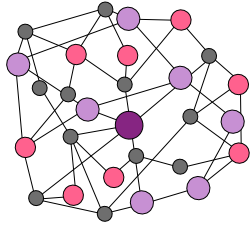
Exponential

Recurrences

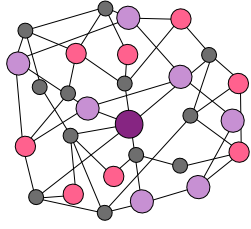
- ♦ Reverse an Array
- ♦ Mergesort
- ♦ Binary Search
- ♦ Selection Sort



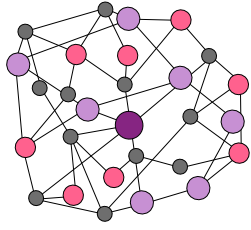
Reverse an Array



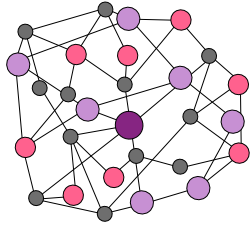
Mergesort



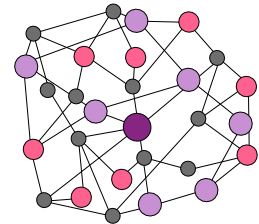
Binary Search



Selection Sort

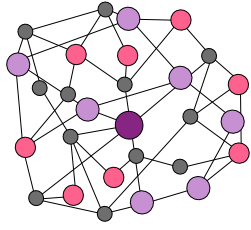


Ordered Maps



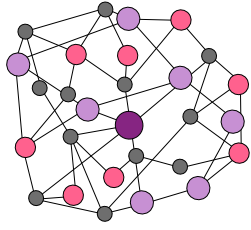
- ♦ Maps: What are they?
- ♦ What makes a map “ordered”?
- ♦ How do we typically implement ordered maps?
 - ♦ What tradeoffs are there with different implementations?

Binary Search Tree



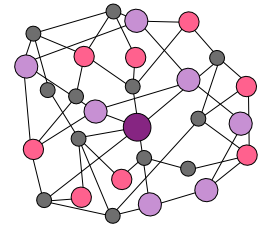
AVL Trees

- ♦ When do we rotate?
- ♦ How do we rotate?

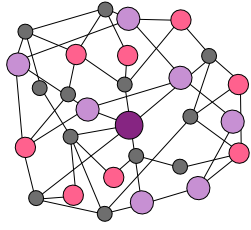


Splay Trees

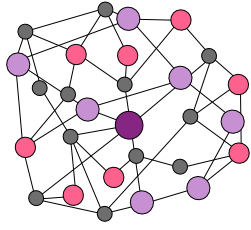
- ♦ When do we splay?
- ♦ How do we splay?



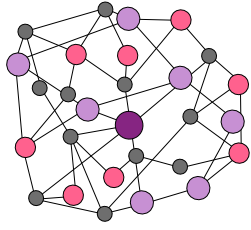
Shortest Paths (Dijkstra)



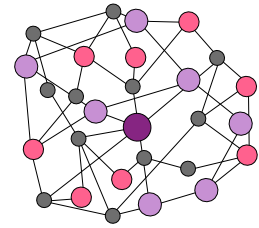
MST (Prim-Jarnik)



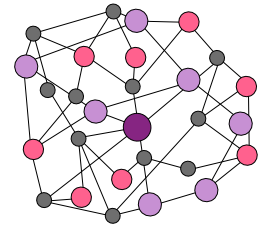
Suffix Arrays



Huffman? Other??



Remainder of the Course



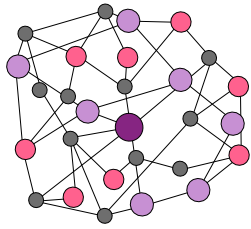
In swotvac, we'll hold an extra exam review session

- ◆ need your input on the format, please see the poll on Ed
- ◆ will run on Thursday 12-2pm here (like usual lectures) on Thursday November 2nd
- ◆ Please post questions/topics that you would like addressed on Ed!

Do not turn up and say “tell me everything you know about algorithms and data structures!!!!” - **you need to study before you come to the session**

Office hours will continue (for all three people who usually attend) this week and next week

Where to from here?



COMP4500/7500

Advanced Algorithms
and Data Structures

COMP4403/7402

Compilers and
Interpreters

COMP3702/7702

Artificial Intelligence

INFS7410

Information Retrieval
and Web Search

COMP3400

Compilers and
Interpreters

COSC3500/7502

High-Performance
Computing

MATH3301

Graph Theory &
Design Theory

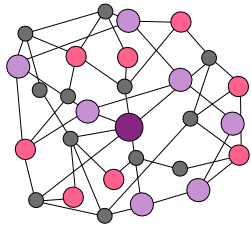
+ a whole lot of others in EECS and SMP

Thesis Projects

https://internal.eait.uq.edu.au/projects/?semester_id=7420&ou=itee

<https://www.itee.uq.edu.au/thesis>

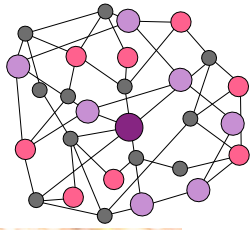
Thanks and Good Luck!



Don't be strangers!

- ◆ You can find me in my office on level 5 most days
- ◆ I like computer science, data structures and algorithms, efficiency problems
- ◆ I like programming
- ◆ I like talking (you know this)
- ◆ I am (probably) recruiting PhD/Hons/Masters, casual RAs for research projects

And a final word from Barry...



COMP3506/7505
STUDENTS

